ACTIVIDAD INTEGRADORA · ELEMENTOS PROGRAMABLES · EQUIPO 2

PWM en RP2040 y RP2350

Control de un motor DC con puente H L298N desde el REPL: registros con punteros, Pico SDK y comparación entre plataformas

INTEGRANTES

Angel Rugerio Jiménez · 201720
Axel García Arellano · 201251

PLATAFORMAS

Raspberry Pi Pico (RP2040)
Raspberry Pi Pico 2 W (RP2350)

AGENDA

Siete partes, unos 48 minutos

#	Parte	Tiempo
1	Introducción y problema tecnológico	5 min
2	Antecedentes, evolución, aplicaciones y arquitectura	10 min
3	Registros y punteros	7 min
4	Pico SDK y opciones de configuración	8 min
5	Comparación RP2040 vs RP2350	5 min
6	Dispositivo externo y comando REPL	5 min
7	Demostración física en las dos tarjetas	8 min

Un pin digital solo da 0 V o 3.3 V. ¿Cómo controlo la velocidad de un motor?

ANTES

Resistencia en serie

El voltaje que sobra se queda en la resistencia o en un transistor en zona lineal y se convierte en **calor**. A media velocidad se desperdicia cerca de la mitad de la energía.

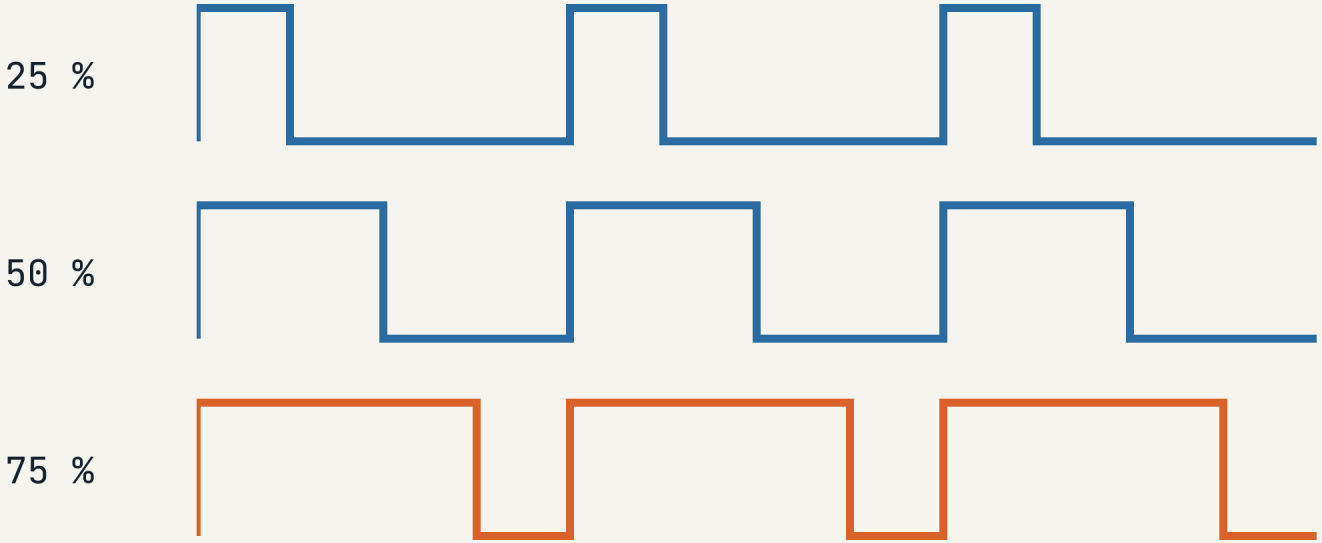
CON PWM

Interruptor todo o nada, muy rápido

El transistor está totalmente encendido (casi sin caída) o totalmente apagado (sin corriente). El motor, por su inercia, solo «ve» el **promedio**. Eficiencia mayor al 90 %.

Y como es un periférico de hardware, el CPU solo escribe un número; la señal se genera sola.

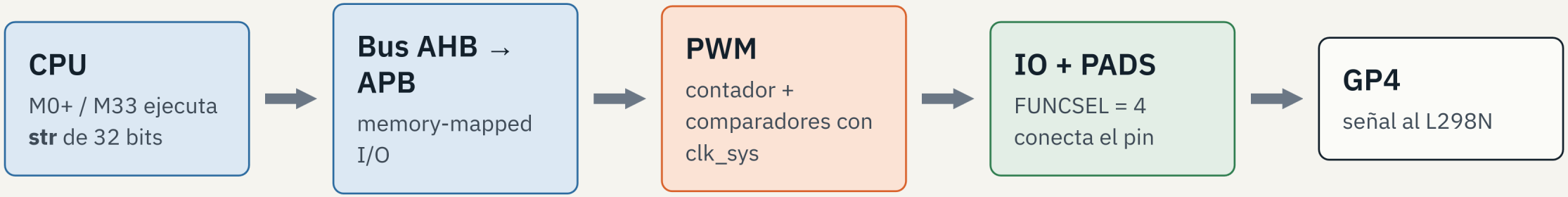
Frecuencia fija, ancho de pulso variable




```
f = 1 / T
D = t_alto / T
V_prom = D · V_alto
```

Mismo periodo T en las tres; solo cambia cuánto tiempo pasa en alto. El motor responde a **V_prom**.

El CPU solo escribe direcciones de memoria



Configurar = escribir CSR, DIV, TOP. **Cambiar la velocidad** = escribir CC. Entre escrituras el CPU no hace nada con el PWM.

Al final de cada periodo (*wrap*) el PWM puede levantar una **interrupción** o pedir una transferencia **DMA**. En este proyecto no se necesitan.

Cómo se controlaba la potencia antes del PWM integrado

Finales s. XIX

Reóstatos en serie: la energía sobrante se vuelve calor

1891

Ward Leonard: motor + generador para variar el voltaje. Caro y pesado

1920 – 1950

Tiratrones y arco de mercurio con control de fase

1957 →

SCR / tiristor: control de ángulo de fase y *choppers* para tracción

Años 70

Timer 555 (1972) y controladores PWM integrados (SG3524, TL494)

Años 80

MOSFET de potencia e IGBT: conmutación rápida y eficiente

Del PWM por software al periférico dedicado

Etapa	Cómo funciona	Mejora
PWM por software	timer + interrupción que prende y apaga el pin	sin hardware extra; usa todo el CPU
Output compare (80s)	comparador de hardware cambia el pin	menos interrupciones, menos jitter
Módulos PWM (90s)	canales dedicados, modo fast y phase-correct	8–10 bits, varios canales
Timers avanzados (2000s)	tiempos muertos, salidas complementarias, DMA	control de motores y energía
RP2040 (2021)	8 slices, cualquier GPIO, divisor 8.4, 16 bits	flexibilidad y precisión
RP2350 (2024)	12 slices, 2 líneas de IRQ, 150 MHz	más canales, más reloj

Mejoras acumuladas: **eficiencia**, **velocidad**, **integración**, **resolución** y **CPU libre**.

02 · APLICACIONES ACTUALES

Donde hay que dosificar energía, hay PWM

Motores DC

Puente H + PWM: robots, drones, bombas. **Este proyecto.**

Servomotores

Pulso de 1–2 ms cada 20 ms codifica el ángulo

Iluminación LED

Atenuación de focos y retroiluminación de pantallas

Fuentes conmutadas

Convertidores buck/boost: cargadores de celulares y laptops

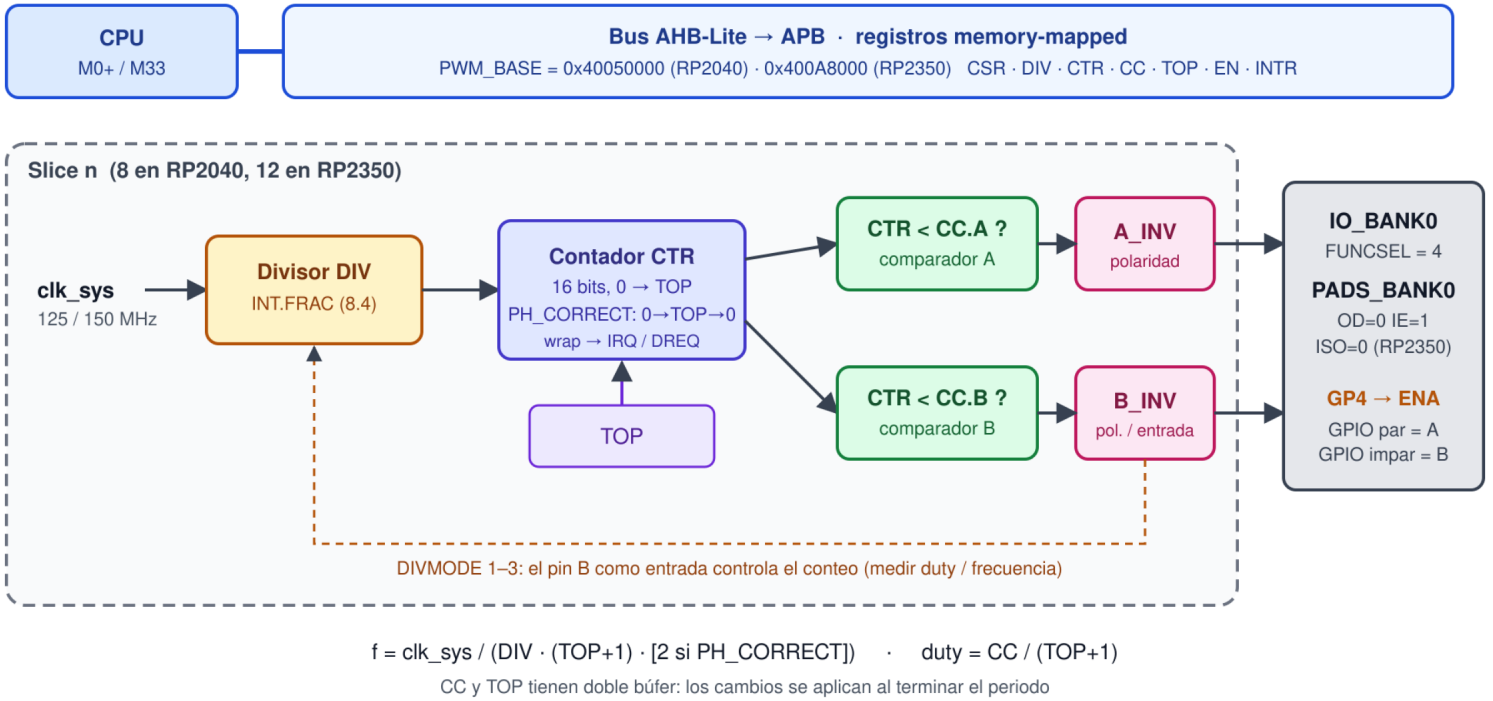
Audio clase D

Bocinas Bluetooth, barras de sonido, audio automotriz

Inversores y VFD

Autos eléctricos, motores industriales, paneles solares

Un slice: divisor, contador, dos comparadores



GP4 es el slice 2, canal A

```
slice = (gpio >> 1) & 7
      = (4 >> 1) & 7 = 2
canal = par → A, impar → B
```

GPIO	Slice	Uso
GP2	1 A	IN1 por SIO
GP3	1 B	IN2 por SIO
GP4	2 A	ENA ← PWM

```
f = clk_sys / (DIV · (TOP+1))
D = CC / (TOP+1)
```

100 Hz, TOP 9999	RP2040	RP2350
clk_sys	125 MHz	150 MHz
DIV	125.0	150.0
CC.A para 70 %	7000	7000

Cinco bloques, cinco direcciones base

Bloque	RP2040	RP2350	Para qué
RESETS	0x4000C000	0x40020000	sacar el PWM de reset
IO_BANK0	0x40014000	0x40028000	FUNCSEL de GP2, GP3, GP4
PADS_BANK0	0x4001C000	0x40038000	habilitar el pad (ISO)
PWM	0x40050000	0x400A8000	generar ENA
SIO	0xD0000000	0xD0000000	IN1 / IN2

Alias atómicos en ambos chips: base + 0x2000 = SET, base + 0x3000 = CLR. Una escritura modifica solo los bits de la máscara.

Offset del slice = 0x14 × 2 = 0x28

Registro	Offset	RP2040	RP2350	Acceso	Valor (100 Hz)
CH2_CSR	0x28	0x40050028	0x400A8028	RW	0x1 (EN = 1)
CH2_DIV	0x2C	0x4005002C	0x400A802C	RW	0x7D0 / 0x960
CH2_CTR	0x30	0x40050030	0x400A8030	RW	lo cuenta el hardware
CH2_CC	0x34	0x40050034	0x400A8034	RW	CC.A = 0 … 10000
CH2_TOP	0x38	0x40050038	0x400A8038	RW	9999 (0x270F)
EN	0xA0 / 0xF0	0x400500A0	0x400A80F0	RW	bit 2 = 1

Cada slice ocupa 5 registros de 32 bits (0x14 bytes). Los offsets internos son iguales en los dos chips; cambia la base y el registro global **EN**.

Un registro, siete campos de control

Bits	Campo	Acceso	Significado	Usamos
0	EN	RW	1 = el slice cuenta	1
1	PH_CORRECT	RW	cuenta arriba y abajo, f / 2	0 · pwm fase
2	A_INV	RW	invierte la salida A	0 · pwm inv
3	B_INV	RW	invierte la salida B	0
5:4	DIVMODE	RW	0 libre · 1 gate B · 2 flanco ↑ B · 3 flanco ↓ B	0
6	PH_RET	SC	retrasa el contador 1 cuenta	0
7	PH_ADV	SC	adelanta el contador 1 cuenta	0

```
csr = CSR_EN | (fase ? CSR_PH_CORRECT : 0) | (inv ? CSR_A_INV : 0);
```

El divisor es fraccional; el comparador es la velocidad

CHn_DIV · punto fijo 8.4

```
[11:4] INT  [3:0] FRAC
DIV = INT + FRAC/16

RP2040 1 kHz: DIV = 12.5
INT = 12, FRAC = 8 → 0xC8
```

Rango 1.0 a 255.94. Fuera de él, el comando **pwm freq** responde con error.

CHn_CC · A [15:0], B [31:16]

Comando	Duty real	CC.A
motor stop	0 %	0
motor 25 …	55 %	5500
motor 50 …	70 %	7000
motor 75 …	85 %	8500
motor 100 …	100 %	10000

Zona muerta: 1–100 % lógico se reparte en 40–100 % real.

De la dirección base al motor




```
/* plataforma.h */
#define PWM_SLICE_REG(s, ofs) (PWM_BASE_ADDR + 0x14 * (s) + (ofs))

/* pwm_regs.c */
static volatile uint32_t *const pwm_cc =
    (volatile uint32_t *)PWM_SLICE_REG(2, PWM_CC_OFS);

*pwm_cc = (*pwm_cc & 0xFFFF0000u) | cc_a;  // conserva CC.B
```

Seis conceptos que hay que poder explicar

Dirección

Cada registro es una posición de memoria: memory-mapped I/O

Puntero

Variable que guarda esa dirección: **uint32_t ***

Desreferencia

***p = v** escribe en la dirección, no en la variable

volatile

El hardware cambia el valor. Sin él, el compilador puede borrar, juntar o reordenar accesos.

Tamaño

Registros de 32 bits, acceso de 32 bits

Máscaras

& aísla, **|** enciende, **& ~** apaga, **<<** lleva al campo

```
while (!(RESET_DONE & (1u << 14))) ; // sin volatile: ciclo infinito
```

Seis pasos antes de que el motor se mueva

#	Bloque	Escritura en pwm_regs.c
1	RESETS	alias CLR ← 1 << 14 (RP2040) o 16 (RP2350); esperar RESET_DONE
2	PADS_BANK0	OD = 0, IE = 1; en RP2350 limpiar ISO (bit 8)
3	IO_BANK0	GPIO4_CTRL.FUNCSEL = 4 (PWM); GPIO2/3 = 5 (SIO)
4	SIO	GPIO_OUT_CLR y GPIO_OE_SET para IN1 / IN2
5	PWM	CSR = 0 → DIV = (INT << 4) | FRAC → TOP = 9999 → CTR = 0 → CSR = EN
6	PWM	CC.A = valor de la velocidad (cada comando)

El orden importa: el pin se conecta al PWM con **CC = 0**, y el slice se apaga mientras cambian DIV y TOP para no generar un pulso a medias.

La misma configuración, en funciones

```
gpio_set_function(PIN_ENA, GPIO_FUNC_PWM);
uint slice = pwm_gpio_to_slice_num(PIN_ENA);    // 2
pwm_config pc = pwm_get_default_config();
pwm_config_set_clkdiv_int_frac4(&pc, div_int, div_frac);
pwm_config_set_wrap(&pc, 9999);
pwm_config_set_phase_correct(&pc, false);
pwm_config_set_output_polarity(&pc, false, false);
pwm_init(slice, &pc, true);
pwm_set_gpio_level(PIN_ENA, 7000);        // 70 %
```

Archivo **pwm_sdk.c**: idéntico para RP2040 y RP2350.

Cada función termina en los mismos registros

Función del SDK	Registros reales	Te ahorra
gpio_set_function()	PADS GPIOx, IO GPIOx_CTRL	offsets e ISO
pwm_gpio_to_slice_num()	— (gpio >> 1) & 7	el mapeo
pwm_config_set_clkdiv_int_frac4()	CHn_DIV INT, FRAC	formato 8.4
pwm_config_set_wrap()	CHn_TOP	—
pwm_config_set_phase_correct()	CSR.PH_CORRECT	máscara
pwm_init()	CSR=0, CTR, CC=0, TOP, DIV, CSR|EN	orden correcto
pwm_set_gpio_level()	CHn_CC mitad A o B	no pisar el otro canal
gpio_put()	SIO OUT_SET / OUT_CLR	offsets por chip

El SDK escribe exactamente los mismos valores

Ventajas del SDK

- Portable: mismo código en los dos chips
- Nombres en vez de números mágicos
- Conoce detalles como ISO y el orden de init

Limitaciones

- Oculta efectos: pwm_init pone CC = 0
- Puede reescribir todo el slice
- Para depurar hay que leer registros igual

```
pwm modo sdk  → motor 50 derecha → pwm regs
pwm modo regs → motor 50 derecha → pwm regs
CSR=0x1 · DIV=0x960 · TOP=0x270F · CC.A=7000 en ambos casos
```

Cada opción se puede probar desde el REPL

Opción	Comando	Registro	Para qué sirve
Frecuencia	pwm freq 1000	DIV	ruido audible vs par a duty bajo
Resolución	pwm top 999	TOP (y DIV)	pasos de duty vs frecuencia máxima
Phase-correct	pwm fase on	CSR.PH_CORRECT	pulsos centrados, f / 2
Polaridad	pwm inv on	CSR.A_INV	cargas activas en bajo
Modo de cuenta	—	CSR.DIVMODE	medir duty o frecuencia en el pin B
Arranque sincronizado	—	EN	varios slices a la vez

Elegimos **100 Hz**: a 1 kHz los pulsos son tan cortos que la corriente en la bobina no sube y el motor no arranca con duty bajo. A cambio, zumba.

Mismo slice, distinto mapa alrededor

Aspecto	RP2040	RP2350
PWM_BASE	0x40050000	0x400A8000
Slices / salidas	8 / 16	12 / 24
EN · INTR	+0xA0 · +0xA4	+0xF0 · +0xF4
Líneas de IRQ del PWM	1 (INTE/INTF/INTS)	2 (IRQ0_*, IRQ1_*)
Bit PWM en RESETS	14	16
Bit ISO en pads	no existe	sí, arranca en 1
SIO OUT_SET · CLR · OE_SET	+0x14 · +0x18 · +0x24	+0x18 · +0x20 · +0x38
clk_sys → DIV a 100 Hz	125 MHz → 125.0	150 MHz → 150.0
CSR, DIV, CTR, CC, TOP	iguales	iguales

Todas las diferencias viven en un archivo

```
/* plataforma.h */
#if PICO_RP2040
#define PWM_BASE_ADDR  0x40050000u
#define RESETS_PWM_BIT 14u
#define PWM_EN_OFS     0x0A0u
#define PADS_TIENE_ISO 0
#elif PICO_RP2350
#define PWM_BASE_ADDR  0x400A8000u
#define RESETS_PWM_BIT 16u
#define PWM_EN_OFS     0x0F0u
#define PADS_TIENE_ISO 1
#endif
```

Con punteros

pwm_regs.c es el mismo; el preprocesador elige las direcciones.

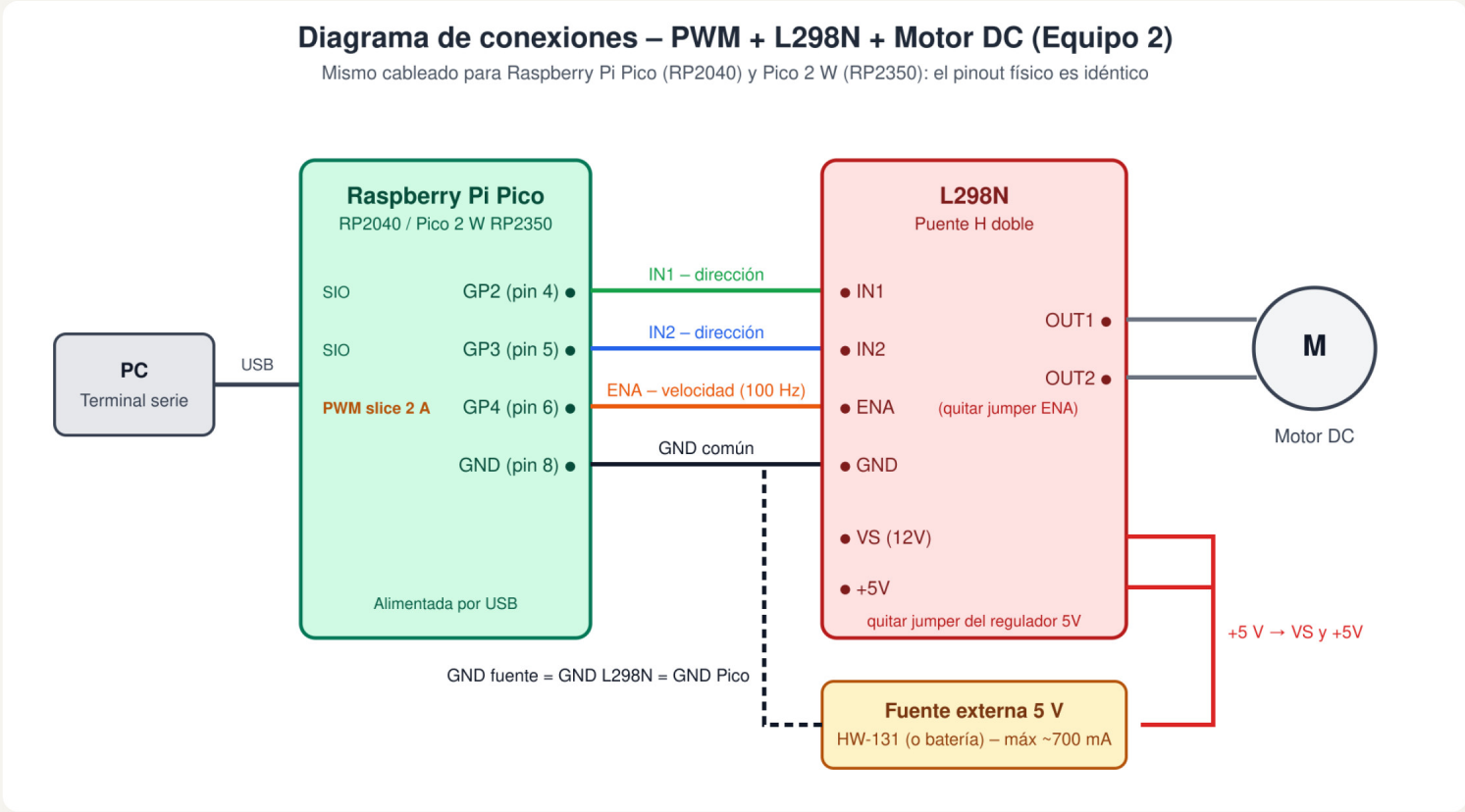
Con SDK

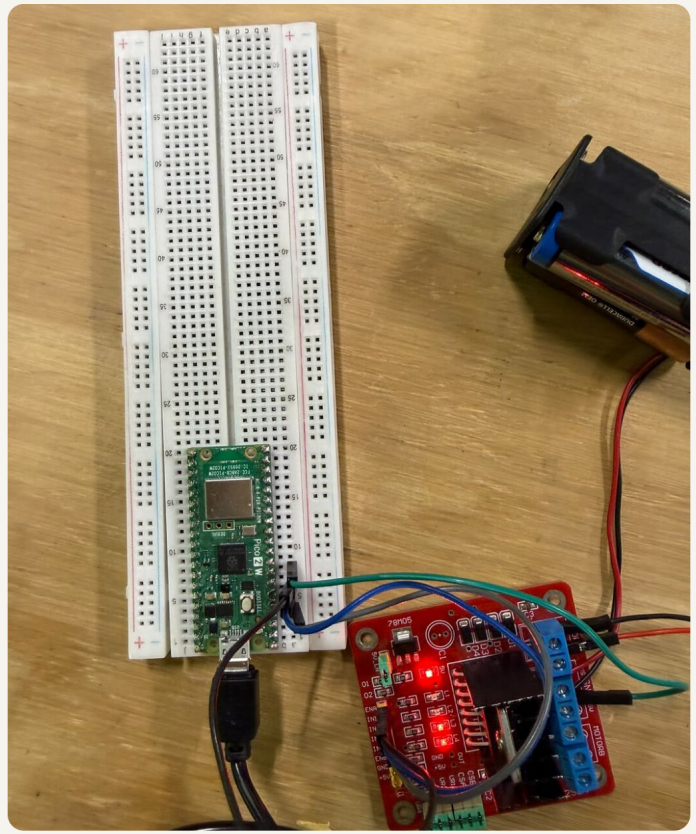
pwm_sdk.c no cambia: el SDK incluye los headers de cada chip.

Al compilar

PICO_BOARD=pico o pico2_w → un .uf2 por tarjeta. Se verifican 30+ direcciones.

Puente H L298N + motor DC





06 · LO QUE PIDIÓ EL MOTOR REAL

Cuatro ajustes medidos en el laboratorio

100 Hz
A 1 kHz el motor no arrancaba con duty bajo. A 100 Hz sí, pero zumba.

40 %
Zona muerta: debajo solo zumba. 1–100 % lógico se reparte en 40–100 % real.

120 ms
Empujón al 100 % al salir de reposo: la fricción estática es mayor que la dinámica.

0 %
Invertir el sentido solo pasando por 0 % con rampa y 500 ms de pausa.

Toda la lógica viene de nuestra Sesión 06 en MicroPython, portada a C en **motor.c**; no sabe si el PWM se maneja con punteros o con SDK.

Una fila más en la tabla de comandos del curso

```
static const command_t commands[] = {
  {"help",  "Muestra la lista…", cmd_help},
  /* on off toggle read sio mask status */
  {"motor", "motor <0-100> <derecha|izquierda>…", cmd_motor},
  {"pwm",   "pwm regs|info|modo|freq|top|fase|inv", cmd_pwm},
  {"demo",  "Secuencia CHALLENGE 06", cmd_demo},
};
```

`motor 50 derecha`

Variable	Valor
argc	3
argv[0]	"motor"
argv[1]	"50" (texto)
argv[2]	"derecha"

read_line() lee por USB hasta Enter → **split_line()** separa con strtok → **execute_command()** busca argv[0] y llama al handler.

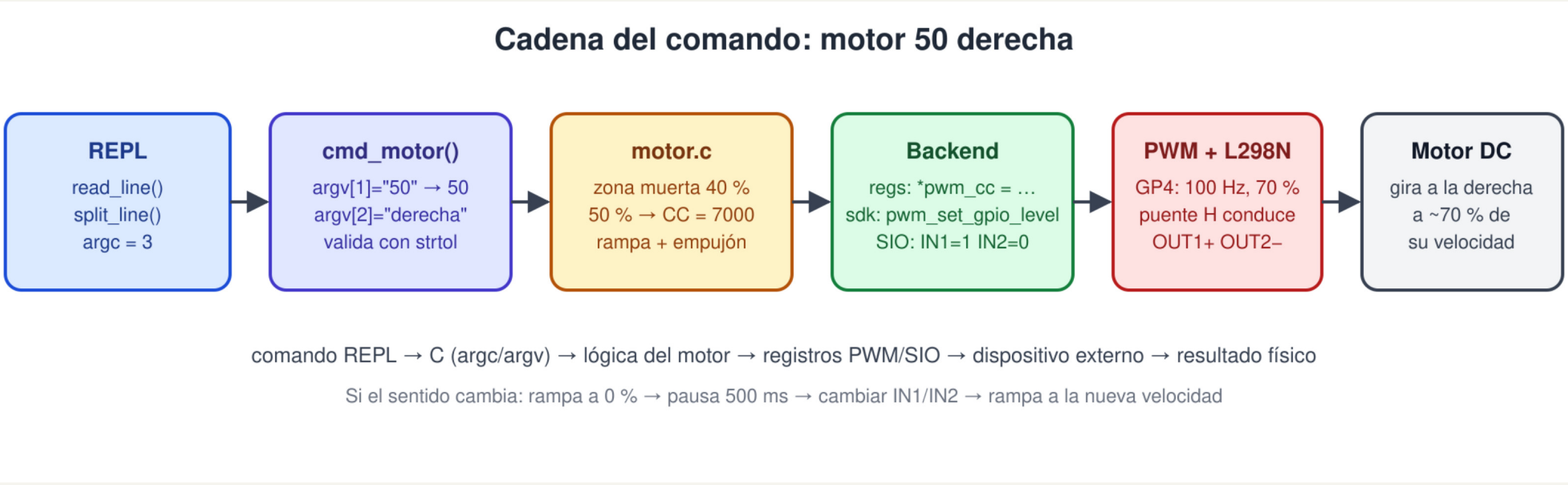
strtol con endptr, no atoi

```
char *fin;
long v = strtol(txt, &fin, 10);
if (fin == txt || *fin != '\0'
    || v < min || v > max)
    return false;

// atoi("5o") = 5, atoi("abc") = 0
// sin avisar del error
```

Entrada	Respuesta
motor 50	muestra el uso
motor 150 derecha	velocidad inválida (0-100)
motor 5o derecha	velocidad inválida
motor 50 arriba	dirección inválida
motor 0 derecha	STOP
pwm freq 10	fuera de rango (DIV)
pwm modo foo	modo inválido

Comando → C → periférico → motor




```
[DIRECCION] DERECHA (FORWARD)  IN1=1 IN2=0
Acelerando: 0 5 10 15 20 25 30 35 40 45 50 %  (CC.A = 7000 de TOP+1 = 10000)
```

07 · DEMOSTRACIÓN FÍSICA

En vivo, en las dos tarjetas

PICO 2 W · RP2350

```
pwm info
motor 50 derecha
pwm regs
motor 30 izquierda  ← [SEGURIDAD]
motor 150 derecha  ← error
pwm freq 1000 · pwm freq 100
pwm modo sdk → motor 50 derecha → pwm regs
demo
```

PICO · RP2040

```
pwm info
motor 50 derecha
pwm regs
demo
```

Comparar: direcciones 0x40050028 vs 0x400A8028, DIV 125 vs 150, misma TOP, mismo CC.A, mismos 100 Hz.

CONCLUSIONES

Lo que nos llevamos

1. El PWM convierte una salida digital en control de potencia eficiente, y como es hardware deja libre al CPU.
2. Configurar un periférico es escribir números en direcciones: base + offset, puntero volatile, máscaras y campos.
3. El SDK escribe los mismos registros; es más portable y legible, pero oculta efectos que hay que conocer.
4. RP2040 y RP2350 comparten el mismo slice, pero cambian bases, offsets, bits de reset, pads y reloj.
5. El hardware real manda: 100 Hz, zona muerta y empujón salieron de medir, no del datasheet.

REFERENCIAS TÉCNICAS

Fuentes

1. Raspberry Pi Ltd. RP2040 Datasheet, cap. 2.14 Resets, 2.19 GPIO, 4.5 PWM
2. Raspberry Pi Ltd. RP2350 Datasheet, cap. 7.5 Resets, 9 GPIO, 12.5 PWM
3. Raspberry Pi Ltd. Raspberry Pi Pico-series C/C++ SDK, y API hardware_pwm (SDK 2.3.1)
4. Raspberry Pi Ltd. Pico y Pico 2 W Datasheets (pinout)
5. STMicroelectronics. L298 Dual Full-Bridge Driver Datasheet
6. Mohan, Undeland, Robbins. Power Electronics: Converters, Applications, and Design. Wiley
7. Rashid, M. H. Electrónica de potencia. Pearson
8. Barr, M. Introduction to Pulse Width Modulation. Embedded Systems Programming, 2001
9. Laboratorio de Elementos Programables, Sesión 06: PWM + puente H L298N + motor DC

RP2350> motor 0 derecha

¿Preguntas?

Código, documentación y evidencias en el repositorio del equipo.